Ders 4 Genişletilmiş Notu: K En Yakın Komşu, Mesafe Ölçüleri ve Normalizasyon

Ders 4 Genişletilmiş Notu: K En Yakın Komşu, Mesafe Ölçüleri ve Normalizasyon

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Derse Giriş: Bilinmeyen Bir Örneği Komşularına Bakarak Sınıflandırmak
- 2. Parametrik ve Parametrik Olmayan Yöntemler
- 3. K En Yakın Komşu Algoritması
- 4. Neden Tek Komşu Yetmez? Kendi Sınıfından Kopmuş Örnekler
- 5. Mesafe Ölçüleri: Minkowski, Manhattan ve Öklid
- 6. Hamming Mesafesi
- 7. Kosinüs Benzerliği
- 8. Hangi Ölçü Ne Zaman Kullanılır?
- 9. Normalizasyon Neden Gerekli?
- 10. Min-Max Normalizasyonu
- 11. Standartlaştırma (Z-Score) ve Gauss Dağılımı
- 12. Kategorik ve Sayısal Özelliklerin Birlikte Olması
- 13. K Değerinin Seçimi
- 14. Özelliklere Ağırlık Verme
- 15. Karmaşıklık ve Tembel Öğrenme
- 16. Voronoi Diyagramı ve Gereksiz Örnekler
- 17. Düzenlenmiş Örnek Tabanlı Öğrenme: Artımlı Silme ve Artımlı Büyütme
- 18. K Boyutlu Ağaç (k-d Tree)

1. Derse Giriş: Bilinmeyen Bir Örneği Komşularına Bakarak Sınıflandırmak

Hoca bu dersin konusunu, K en yakın komşu (k-Nearest Neighbors, k-NN) yöntemini, slayt yerine ekrana yazarak anlattı. Bu tür konularda yazarak anlatmanın daha faydalı olduğunu düşündüğünü, slaytı daha çok görüntünün ön planda olduğu konularda kullanacağını söyledi.

Ders basit bir çizimle başladı. İki özellikli, yani x_1 ve x_2 özellikleri olan bir örnek kümesi düşünelim. Bu örnekler iki sınıfa ait olsun: A ve B. Hepsi etiketli verilerdir; hangi sınıfa ait oldukları bilinir. Sonra bu düzleme sınıfı bilinmeyen yeni bir örnek konur.

Hoca öğrencilere sordu: Bu örnek hangi sınıfa aittir? Öğrenciler “A sınıfı” diye cevap verdi. Neden? Çünkü bilinmeyen örneğin x_1 ve x_2 değerleri, genel olarak A sınıfının örneklerine daha yakın bir yerde duruyor. Aynı şekilde B örneklerinin arasına konan bir bilinmeyen örnek için de hemen “B sınıfıdır” deriz: A sınıfından oldukça uzakta, B sınıfına ise oldukça yakındır.

Bu sezgi, bugünkü yöntemin özüdür: Bir örneğin sınıfı, ona en çok benzeyen örneklerin sınıfına bakılarak belirlenir.

2. Parametrik ve Parametrik Olmayan Yöntemler

Hoca, önceki haftalarda gördüğümüz yöntemlerle bugünkü yöntemi karşılaştırdı.

Karar ağacında (decision tree) özelliklerin üzerinden ilerleyerek, eğitim örneklerinin istatistiksel durumuna bakıp bilinmeyen bir örneğin nasıl sınıflandırılacağını gördük. Bayes teoreminde de benzer bir yapı vardı: Eğitim örneklerinden elde edilen olasılıklar elimizdeydi. Bu olasılıklarla bir model kurduk ve bilinmeyen örnek geldiğinde hangi sınıfa daha benzer olduğuna bu modelle karar verdik.

Bu tür yöntemlere **parametrik yöntemler** denir. Eğitim örneklerinden, parametrelerden faydalanarak bir model oluştururlar. Bayes ve karar ağacı bu gruptandır.

K en yakın komşu ise **parametrik olmayan (non-parametric)** bir yöntemdir. Burada eğitim aşamasında hiçbir şey yapılmaz. Yapılan tek iş, etiketli örneklerin hepsini biriktirmektir. Bütün iş tahmin (test) sırasında yapılır: Verilen bir K değeri için, bilinmeyen örneğin en çok benzediği K örnek bulunur.

> **Ek bilgi (derste anlatılmadı):** Buradaki "parametrik olmayan" ifadesi, modelin eğitimden önce sabit sayıda parametreyle özetlenmediği anlamına gelir. k-NN'de "model" eğitim verisinin kendisidir; veri büyüdükçe model de büyür.

3. K En Yakın Komşu Algoritması

Elinizde n tane eğitim örneği olsun: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Burada x_i örneğin özellik vektörü, y_i sınıf etiketidir. Bu örnekler üzerinde eğitim adına hiçbir işlem yapılmaz.

Bilinmeyen bir x örneği geldiğinde şu adımlar uygulanır:

1. x örneğinin bütün eğitim örneklerine mesafesi (ya da benzerliği) hesaplanır.
2. Mesafeler küçükten büyüğe sıralanır. En küçük mesafe, en yakın yani en benzer örnektir.
3. En benzer K örnek seçilir.
4. Bu K örnek arasında çoğunluk oylaması (majority voting) yapılır. En çok örneğe sahip sınıf, bilinmeyen örneğin sınıfı kabul edilir.

Örnek 1: K = 3 ile çoğunluk oylaması

- K = 3 alınsın. Bilinmeyen x örneğine en çok benzeyen üç örnek x_2, x_5 ve x_9 olsun.
- x_2 A sınıfından, x_5 B sınıfından, x_9 A sınıfından.
- Üç komşudan ikisi A olduğu için çoğunluk A'dadır. Bilinmeyen örnek A sınıfına atanır.

Bu algoritmada iki temel karar vardır:

- **K kaç seçilecek?**
- **Benzerlik (mesafe) nasıl ölçülecek?**

Dersin geri kalanı bu iki sorunun etrafında ilerledi.

4. Neden Tek Komşu Yetmez? Kendi Sınıfından Kopmuş Örnekler

Hoca neden tek bir komşuya değil, K komşuya bakıldığını yine çizimle gösterdi.

İlk çizimdeki A örneklerine yakın duran bilinmeyen örneğin hemen yanına, mavi renkli tek bir B örneği konduğunu düşünelim. Size "bu bilinmeyen örnek kime daha çok benziyor?" diye sorulsa, belki "hemen yanımda bir B örneği var, o zaman B'dir" dersiniz. Bu olabilecek bir durumdur. Bu B örneği bir parazitten (gürültüden) kaynaklanıyor olabilir. Ya da gerçekten B sınıfının, kendi sınıfının genel örneklerinden uzakta, A sınıfına yakın duran bir örneği olabilir.

- K = 1 alınırsa, yani yalnızca en benzer örneğe bakılırsa, bilinmeyen örnek B sınıfına atanır.
- K = 3 alınırsa çoğunluk A olur ve örnek A sınıfına atanır.

K = 3 durumu daha mantıklıdır. Çünkü bilinmeyen örnek genel olarak A sınıfına yakındır. Oradaki B örneği aslında kendi sınıfından uzaklaşmış bir örnektir.

Hoca ikinci bir çizimde bu kez iki tane "kaçmış" B örneği koydu. Bu durumda K = 3 için yine mavi (B) kazanıyordu. Fakat K = 5 alındığında en yakın beş komşunun üçü kırmızı (A), ikisi mavi (B) oluyordu ve örnek A'ya atanıyordu. Gözle bakıldığında da bilinmeyen örneğin A

sınıfına daha çok benzediği görülüyordu. Hoca iki boyutlu uzayın bu yüzden seçildiğini söyledi: Yalnızca iki özellik varken her şeyi görsel olarak değerlendirmek çok kolaydır.

K en yakın komşulukta yapılmaya çalışılan budur: Öyle uygun bir K değeri belirlenmelidir ki, kendi sınıfının çok da yakınında durmayan eğitim örnekleri olsa bile doğru sınıflandırma yapılabilsin. Bilinmeyen örnek, tek tek örneklere göre değil, sınıfların genel karakteristiğine göre sınıflandırılsın.

> **Hocanın vurgusu:** Makine öğrenmesinde çok fazla yöntem vardır. Hoca ilk derste dönem içinde en başarılı olanları anlatacağını söylemişti. K en yakın komşu da bu açıdan en başarılı yöntemlerden biridir.

5. Mesafe Ölçüleri: Minkowski, Manhattan ve Öklid

Hoca K'nin seçimine geçmeden önce mesafe hesabıyla başladı. Benzerliği ölçmek için mesafe yöntemleri kullanılır.

5.1 Minkowski mesafesi

Bir örneğin D tane özelliği olsun. Eğitim örneğini x, bilinmeyen örneği x_{new} ile gösterelim. Hoca, y harfini sınıf etiketi için kullandığımızdan karışmasın diye bilinmeyen örneği x_{new} olarak yazdı. x_i, örneğin i. özelliğidir.

Minkowski mesafesi şöyledir:

$$d(x, x_{new}) = \left(\sum_{i=1}^{D} |x_i - x_{new,i}|^p \right)^{1/p}$$

- $p = 1$ ise bu **Manhattan mesafesi**dir.
- $p = 2$ ise bu **Öklid (Euclidean) mesafesi**dir.

Hoca bu ölçüleri büyük ihtimalle daha önce gördüğümüzü, ama herkesin notlarında mutlaka bulunsun diye bir kez daha anlattığını söyledi.

5.2 Manhattan mesafesi

$$d_{Man}(x, x_{new}) = \sum_{i=1}^{D} |x_i - x_{new,i}|$$

Bütün özelliklerdeki farkların mutlak değerleri toplanır. Mutlak değer alınmasının sebebi şudur: Farkın negatif ya da pozitif olması bizim için önemli değildir. Yalnızca mesafenin büyüklüğünü anlamaya çalışıyoruz.

Izgara (grid) benzeri bir yapınız varsa Manhattan mesafesi kullanılır. İki boyutta düşünürsek, bir noktadan diğerine kuş uçuşu değil, önce yatay sonra dikey kenarlar boyunca gidilerek ölçülen mesafedir.

5.3 Öklid mesafesi

$$d_{Euc}(x, x_{new}) = \sqrt{\sum_{i=1}^{D} (x_i - x_{new,i})^2}$$

Bütün özelliklerdeki farkların kareleri alınır, toplanır ve toplamın karekökü alınır.

Öklid mesafesi Manhattan'dan daha hassastır ve genellikle K en yakın komşulukta Öklid mesafesi kullanılır. Ancak işlem yükü daha fazladır: Manhattan'da yalnızca mutlak değer alınırken, burada kare alma, toplama ve karekök vardır. Hız sizin için önemliyse ve çok hassas bir ayrıma ihtiyacınız yoksa Manhattan mesafesi zaman açısından daha verimlidir.

Örnek 2: Aynı iki örnek için Manhattan ve Öklid mesafesi

- Üç özellikli iki örnek: $x = (3, 5, 1)$ ve $x_{new} = (2, 1, 6)$.
- Manhattan:

$$|3-2| + |5-1| + |1-6| = 1 + 4 + 5 = 10$$

- Öklid:

$$\sqrt{(3-2)^2 + (5-1)^2 + (1-6)^2} = \sqrt{1 + 16 + 25} = \sqrt{42} \approx 6{,}48$$

- Hoca derste $\sqrt{42}$ için önce "6,6 falan" dedi, sonra 36 ile 49 arasında olduğunu düşünerek "6,4 falan etmesi lazım" diye düzeltti. Doğru değer yaklaşık 6,48'dir.

K en yakın komşulukta bu hesap, bilinmeyen örnek ile bütün eğitim örnekleri arasında tek tek yapılır. Burada bir benzerlik değil, bir mesafe hesaplanmaktadır. Bu yüzden mesafeler küçükten büyüğe sıralanır ve en küçük K tanesi seçilir.

6. Hamming Mesafesi

Hamming mesafesi biraz daha farklıdır ve K en yakın komşulukta çok kullanılan bir ölçü değildir. Genellikle kategorik özellikler için kullanılır. Özellikler tek tek karşılaştırılır.

Hoca derste şu kuralı verdi: İki örnekte bir özellik aynıysa o özellik için değer 1, farklıysa 0'dır. Sonra bu değerler toplanır. (Bu tanımın standart Hamming mesafesinin tersi olduğuna Ders Sonrası Notlar'da değinilmiştir.)

Örnek 3: Şehir özellikleriyle Hamming (derste anlatıldığı haliyle)

- x = (Ankara, İzmir, İstanbul), x_{new} = (Ankara, Diyarbakır, Muğla).
- Birinci özellik eşit, ikinci ve üçüncü farklı: (1, 0, 0).
- Hoca toplamı 1 olarak verdi.

Bir öğrenci, ikinci özellik de eşit olsaydı ne olacağını sordu.

Soru: İkinci örnekte Diyarbakır yerine İzmir olsaydı mesafeyi nasıl gösterecektik? **Cevap:** x = (Ankara, İzmir, İstanbul) ve x_{new} = (Ankara, İzmir, Muğla) olursa değerler (1, 1, 0) olur ve toplam 2 çıkar. Yani burada yapılan, kaç tane özelliğin benzer olduğunu toplamaktır.

Başka bir öğrenci, sayısal özelliklerde bu ölçünün işe yarayıp yaramayacağını sordu.

Soru: Sayısal özelliklerde örneklerin hiçbir özelliği tam olarak aynı olmuyor, üst üste binmiyorlar. Hamming bu yüzden yalnızca bu tarz (kategorik) durumlarda mı kullanılıyor? **Cevap:** Evet, Hamming genel olarak kategorik özelliklerde kullanılır. Kategorik özelliklerin değerlendirilmesiyle ilgili başka yöntemler de ileride anlatılacak; Hamming bunun en temel şeklidir.

7. Kosinüs Benzerliği

Kosinüs benzerliği (cosine similarity) genellikle doküman sınıflandırmada kullanılır, başka alanlarda da kullanılabilir. Burada iki örneğin yakınlığı, onları birer vektör olarak düşünüp aralarındaki açıya bakarak ölçülür.

7.1 Neden açıya bakılır?

Hoca doküman sınıflandırmadan bir örnek verdi. A dokümanında bir kelime 20 kez, B dokümanında aynı kelime 50 kez geçsin. Bu sayılarla doğrudan mesafe hesaplanırsa, iki doküman birbirinden çok uzakmış gibi görünebilir. Oysa belki B dokümanı daha uzun olduğu için o kelime 50 kez, A daha kısa olduğu için 20 kez geçmiştir. Burada önemli olan mesafenin büyüklüğü değil, benzerliktir. Kosinüs benzerliği, iki vektörün yönlerinin ne kadar yakın olduğunu ölçtüğü için bu sorunu ortadan kaldırır.

Kosinüs benzerliği iki vektörün iç çarpımı (dot product) ile yazılır:

$$\cos\theta = \frac{x \cdot x_{new}}{\|x\|\,\|x_{new}\|} = \frac{\sum_{i=1}^{D} x_i\, x_{new,i}}{\sqrt{\sum_{i=1}^{D} x_i^2}\,\sqrt{\sum_{i=1}^{D} x_{new,i}^2}}$$

7.2 Sözlük eğitim verisinden çıkarılır

Hoca, Naive Bayes'te yaptığımız gibi, önce kelimelerden bir sözlük (vocabulary) çıkarıldığını hatırlattı. Her doküman, sözlükteki her kelimenin o dokümanda kaç kez geçtiğini gösteren bir vektöre dönüşür.

> **Hocanın vurgusu:** Sözlük her zaman eğitim örnekleri için baştan çıkarılır. İşlem (test) yaparken asla sözlük çıkarmayın. Bunu söyleyince "insan niye işlem yaparken sözlük çıkarsın" gibi gelebilir, ama bu gerçekten çok sık yapılan bir hatadır.

7.3 Örnek hesap

Örnek 4: İki doküman parçası için kosinüs benzerliği

- Hoca ekrana iki doküman parçası yazdı. Birincisi "Data, digital economy … new oil" benzeri bir cümleydi.
- Önce iki dokümandan sözlük çıkarıldı, sonra her doküman sözlüğe göre kelime sayısı vektörüne çevrildi. Birini x, diğerini x_{new} olarak aldı.
- İç çarpım: karşılıklı özelliklerin çarpımlarının toplamı. Derste $x \cdot x_{new} = 3$ bulundu.
- $\|x\|$: x vektörünün bileşenlerinin kareleri toplanıp karekökü alınır. x dokümanında bir kelime iki kez geçiyordu ($2^2 = 4$), diğer kelimeler birer kez. Hoca bu toplamı sayarak 10 buldu:

$$\|x\| = \sqrt{10}$$

- $\|x_{new}\|$: x_{new} vektöründe dört tane 1 vardı:

$$\|x_{new}\| = \sqrt{4} = 2$$

- Derste bulunan sonuç:

$$\cos\theta = \frac{3}{2\sqrt{10}} \approx 0{,}47$$

Bilinmeyen dokümanın sınıfını bulmak için yine K en yakın komşu kullanılır. Bu kez noktalar arasındaki mesafeyi Manhattan ya da Öklid ile ölçmek yerine, bütün eğitim dokümanlarıyla kosinüs benzerliği hesaplanır. Sonra en benzer K doküman alınır ve çoğunluk oylaması yapılır.

7.4 Aradan sonra gelen soru

Teneffüsten sonra bir öğrenci kosinüs benzerliğini tam anlamadığını söyledi.

Soru: Kosinüs benzerliğini kullanmamızın sebebi, iki örneğin boyutunun (uzunluğunun) farklı olması mı? **Cevap:** Tam olarak yalnızca o değil, ama o da etkenlerden biri. Öklid mesafesi gerçekten mesafenin kaç olduğunu, yani büyüklüğü (magnitude) ölçer. Kosinüs benzerliği ise iki vektörün birbirine ne kadar benzediğini açı olarak ölçer. Bu yüzden bir kelimenin bir dokümanda 5, diğerinde 10 kez geçmesinden etkilenmez. Öklid ile ölçseydiniz büyüklüklerin farkı alındığı için iki doküman gerçekten farklıymış gibi görünecekti. Kosinüs benzerliği büyüklüklerin farkına değil, vektör normları üzerinden benzerliğe bakar. Bu yüzden dokümanlarda benzerliği daha iyi yakalar.

Soru: Peki verileri normalize edip Manhattan (ya da Öklid) kullansaydık? **Cevap:** Normalize etseniz de durum değişmezdi. Dokümanlarda, en temel haliyle yalnızca bir benzerlik ölçerek k-NN kullanacaksanız kosinüs benzerliği kullanmanız gerekir. Doğal dil işlemede doküman

sınıflandırma için başka birçok yöntem görülecektir; örneğin Naive Bayes doküman sınıflandırmada kullanılan temel yöntemlerden biridir.

Hoca derste doğal dil işleme dersi alan olup olmadığını da sordu; öğrenciler henüz o konuya gelinmediğini söyledi.

7.5 Mahalanobis mesafesi

Hoca bunlardan başka Mahalanobis mesafesini de andı. Bu mesafe kovaryans bilgisini de kullanır. Fakat K en yakın komşulukta tercih edilen bir yöntem olmadığı için ayrıntısına girmedi; ileride başka bir noktada değinilebileceğini söyledi.

8. Hangi Ölçü Ne Zaman Kullanılır?

Hoca mesafe ölçülerini şöyle özetledi:

1. **Öklid mesafesi:** K en yakın komşulukta ideal mesafe ölçüsüdür.
2. **Manhattan mesafesi:** Daha hızlı olmasını istiyorsanız ve çok hassas bir ayrıma ihtiyacınız yoksa kullanılır. "Çok hassas olmamak" şu demektir: Örnekler zaten güzel yerleşmiştir, birbirine çok yakın farklı sınıftan örnekler yoktur ve karışıklık olacağından endişe etmiyorsunuzdur.
3. **Hamming mesafesi:** Kategorik özellikleriniz varsa kullanılır.
4. **Kosinüs benzerliği:** Doküman türü bilgileri sınıflandırmak istiyorsanız kullanılır.

Bu noktada derse yaklaşık 15 dakika ara verildi.

9. Normalizasyon Neden Gerekli?

Aradan sonra hoca, öğrencinin sorduğu normalizasyon konusunu k-NN'in K seçimine geçmeden önce anlattı.

Elimizde öğrencilerin bilgileri olsun ve öğrencileri benzerliklerine göre gruplamak isteyelim. Hoca bilerek "enteresan, anlamsız" özellikler seçti: not ortalaması (GPA), yaş, kilo ve boy.

Bu özelliklerin değer aralıkları çok farklıdır:

- **GPA:** 0 ile 4 arasında değişir. Örnek değerler 3,2 ve 2,8.
- **Yaş:** örnek değerler 29 ve 21.
- **Kilo:** örnek değerler 57 ve 95.
- **Boy:** örnek değerler 152 ve 195 (cm).

Her özellikte "büyük fark" kavramı farklıdır. GPA 4 üzerinden değerlendirildiği için 0,4'lük bir oynama bile çok büyük bir farktır. Yaşta bir iki yaşlık fark çok önemli değildir. Boyda da 193 cm ile 192 cm arasındaki fark önemsizdir, ama 193 cm ile 180 cm arasındaki fark önemlidir. Yani benzerlik ölçülürken her özelliğin önemi farklıdır.

Örnek 5: Normalize edilmemiş verinin Öklid mesafesi

- $x_1 = (3{,}2;\ 29;\ 57;\ 170)$ ve $x_2 = (2{,}3;\ 28;\ 79;\ 160)$ olsun (GPA, yaş, kilo, boy).
- Öklid mesafesi:

$$d(x_1, x_2) = \sqrt{(3{,}2-2{,}3)^2 + (29-28)^2 + (57-79)^2 + (170-160)^2}$$

$$= \sqrt{0{,}81 + 1 + 484 + 100} = \sqrt{585{,}81} \approx 24{,}2 \quad \text{(ara adım eklendi)}$$

- GPA'daki 0,9'luk fark, 4 üzerinden değerlendirilen bir özellik için çok büyüktür. Ama toplamın içinde büyük değerli özelliklerin farklarının yanında "kaynayıp gider". Hesap bu iki öğrenciyi, GPA'daki ciddi farka rağmen, büyük özellikler birbirine yakınsa birbirine çok benzer sayar. Oysa topu topu dört özellik vardır ve birinde ciddi bir fark olduğu gözle görülmektedir.

  Ek bilgi (derste anlatılmadı): Bu örnekte GPA farkının kareler toplamına katkısı $0{,}81/585{,}81 \approx \%0{,}14$'tür. Yani mesafeyi neredeyse tamamen kilo ve boy belirler.

Hoca bunu ikinci bir örnekle de anlattı: Bir öğrencide GPA farkı 1,8 gibi çok büyük, ağırlık farkı (170 ile 172) ise yalnızca 2 olsun; başka bir öğrencide GPA farkı küçük ama başka bir özellikteki fark 10 olsun. Toplam mesafeye bakıldığında 1,8'lik büyük GPA farkı yine kaybolup gider.

Bu durumdan kurtulmak için mesafe ölçümü kullanan yöntemlerde, önce **normalizasyon** ya da **standartlaştırma** ile bütün özellikler eşit aralığa getirilir.

> **Hocanın vurgusu:** Veriyi kullanmadan önce bütün eğitim örneklerini normalizasyon ya da standartlaştırma ile eşit aralıklara getirmeniz gerekir. Sonra test edeceğiniz örneği de işleme sokmadan önce aynı normalizasyondan ya da standartlaştırmadan geçirmeniz gerekir. Bu mutlaka yapmanız gereken bir şeydir.

10. Min-Max Normalizasyonu

Normalizasyonda iki genel yaklaşım vardır.

Birincisi: [0, 1] aralığına min-max normalizasyonu. Elinizdeki eğitim örneklerinde her özellik için o özelliğin minimum ve maksimum değeri bulunur. Sonra her özellik kendi minimum ve maksimumuna göre normalize edilir:

$$x'_i = \frac{x_i - \min_i}{\max_i - \min_i}$$

Böylece her özellik 0 ile 1 arasına gelir. GPA da, boy da 0-1 aralığına çekildiği için hepsi oransal olarak aynı ölçeğe gelir ve etkileri daha net anlaşılır. 3,2 ile 2,3 arasındaki farkın etkisi artık boydaki farkın içinde kaybolmaz.

İkincisi: [-1, 1] aralığına normalizasyon. Veriyi sıfır merkezli yapmak için kullanılır. Hoca genelde [0, 1] aralığının kullanıldığını, ama bir yerlerde karşımıza çıkabileceği için [-1, 1]'i de gösterdiğini söyledi.

$$x'_i = 2 \cdot \frac{x_i - \min_i}{\max_i - \min_i} - 1$$

11. Standartlaştırma (Z-Score) ve Gauss Dağılımı

Özellikler arasındaki ölçek farkını ortadan kaldırmanın ikinci yolu **standartlaştırma** (standardization) ya da **z-score normalizasyonu**dur. Her örnekten ortalama çıkarılır ve standart sapmaya bölünür. Böylece her özelliğin ortalaması 0, standart sapması 1 olur.

N tane eğitim örneğimiz olsun. $x_{d,i}$, d. eğitim örneğinin i. özelliğidir.

1. Her özellik için ayrı ayrı ortalama alınır:

$$\mu_i = \frac{1}{N} \sum_{d=1}^{N} x_{d,i}$$

2. Her özellik için standart sapma hesaplanır:

$$\sigma_i = \sqrt{\frac{1}{N} \sum_{d=1}^{N} (x_{d,i} - \mu_i)^2}$$

3. Her özelliğin yeni değeri: eski değer eksi ortalama, bölü standart sapma.

$$x'_{d,i} = \frac{x_{d,i} - \mu_i}{\sigma_i}$$

Örnek 6: Özellik başına ortalama

- İki eğitim örneği olsun: $(1, 3, 5)$ ve $(2, 4, 6)$.
- Ortalama her özellik için ayrı alınır:

$$\mu = \left(\frac{1+2}{2},\ \frac{3+4}{2},\ \frac{5+6}{2}\right) = \left(\frac{3}{2},\ \frac{7}{2},\ \frac{11}{2}\right)$$

Ek bilgi (derste anlatılmadı): Aynı örnekte her özelliğin standart sapması $\sigma = 0{,}5$ çıkar. Standartlaştırılmış değerler birinci örnek için $(-1, -1, -1)$, ikinci örnek için $(1, 1, 1)$ olur.

11.1 Hangisi ne zaman?

Hoca şu kuralı verdi: Verinin dağılımı Gauss (normal) dağılımıysa standartlaştırma, değilse min-max normalizasyonu kullanılır.

Gauss dağılımı ne demektir? Gerçek dünyada pek çok olay Gauss dağılımına uyar. Hoca sınıftaki öğrencilerin boyunu örnek verdi. 40 öğrencili bir sınıfta çoğunluk birbirine yakın boylardadır. Kız öğrencilere bakıyorsanız çoğunun boyu belki 1,58 ile 1,63 m arasındadır. Az sayıda daha kısa, yine az sayıda da 1,70-1,75 m gibi daha uzun öğrenci vardır. Büyük bir grubun yaklaşık benzer değerlerde, küçük bir grubun uçlarda olması Gauss dağılımıdır.

Verinizin Gauss dağılımında olduğunu biliyorsanız daha hassas normalizasyon şekli standartlaştırmadır. Ama dağılımın ne olduğunu her seferinde bilmek zorunda değilsiniz. Bilmiyorsanız ve bununla uğraşmak istemiyorsanız şunu yaparsınız: Veriyi bir kez min-max normalizasyonuyla normalize edip başarıyı ölçersiniz, bir kez de standartlaştırmayla normalize edip başarıyı ölçersiniz. Hangisi daha iyiyse onu kullanırsınız.

12. Kategorik ve Sayısal Özelliklerin Birlikte Olması

Soru: Girdinin özelliklerinden biri kategorik, diğerleri sürekli (sayısal) olabilir. O zaman sayısal olanları normalize edip kategoriği ayrı mı bırakıyoruz? Böyle örnekler oluyor mu? **Cevap:** Güzel bir soru. Evet, böyle kullanılıyor. Kategorik özellikleri hiç ellemeyip Hamming mantığıyla 0-1 olarak kullandığınız da oluyor. Fakat k-NN gibi yöntemler genellikle sayısal değerli özelliklerle kullanılır. Doküman sınıflandırmada bile aslında kelimelerin kaçar kez kullanıldığını saydık. Genelde iki tür birbirine çok karışmaz, ama karışırsa kategorikler için 0-1 kullanılır.

Hoca bu konuda şu tavsiyeyi verdi:

> **Hocanın vurgusu:** Bu yöntemleri uygularken sizin veriniz ve sizin probleminiz nedir, buna bakarak karar vermeniz çok önemlidir.

- Özelliklerin çoğu kategorikse, ileriki haftalarda anlatılacak one-hot encoding gibi yöntemlerle kategorik özellikler sayıya çevrilip sayılarla çalışılabilir.
- Az sayıda kategorik özellik varsa, nasıl değerlendirileceğine şartlarınıza göre karar verirsiniz. Bunun kesin bir reçetesi yoktur. Öğrencinin dediği gibi, az sayıda kategorik özellik Hamming mantığıyla 0-1 olarak mesafeye eklenebilir.

13. K Değerinin Seçimi

13.1 K'nin aralığı

Öncelikle K'nin tek sayı olması iyidir: 1, 3, 5, 7 gibi.

> **Ek bilgi (derste anlatılmadı):** Tek sayı seçilmesinin sebebi, iki sınıflı bir problemde çoğunluk oylamasında eşitlik çıkmasını önlemektir. Üç ve daha fazla sınıfta eşitlik yine çıkabilir; o zaman örneğin en yakın komşunun sınıfı seçilir.

K'nin çok küçük olması iyi değildir. Hoca yine iki özellikli bir çizim yaptı. Bilinmeyen örnek için K = 1 alındığında örnek kırmızı sınıfa daha çok benziyordu, ama K = 3 alındığında maviye daha çok benzeyecek gibiydi. K'nin 1 gibi çok küçük olması parazit açısından uygun değildir: Elinizde parazitli bir örnek ya da örnek grubu varsa hemen yanlış sınıflandırma yapılabilir.

K'yi çok büyük almak da iyi değildir. K'yi 9, 11 gibi değerler aldığınızda komşuluk dağılmaya başlar: Aslında çok da benzemeyen örnekler işin içine girer ve sınıflandırma başarısını bozmaya başlar. Hocaya göre K genelde 7'yi geçince sonuçlar bozulur; bu yüzden K'nin 1 ile 7 arasında olması iyidir.

13.2 En iyi K nasıl bulunur?

En iyi K'yi bulmak için, karar ağacında da gördüğümüz gibi, veri üç parçaya ayrılır: eğitim (training), doğrulama (validation) ve test (testing). k-NN'de aslında eğitim yapılmaz, ama yine de bir eğitim verisi vardır.

Örnek 7: Doğrulama kümesiyle K seçimi

- $D_1, D_2, \ldots, D_{14}$ eğitim verisi, D_{15}'ten D_{20}'ye kadar doğrulama verisi olsun.
- Doğrulama verisindeki örneklerin her biri, eğitim verisi kullanılarak k-NN ile sınıflandırılır. Bu, K = 1 için yapılır ve başarı ölçülür.
- Aynı işlem K = 3, K = 5, K = 7 için tekrarlanır. K = 9 da denenebilir, ama 9'u geçtikten sonra genellikle iyi bir başarı elde edilmez.
- En yüksek başarıyı veren K seçilir.
- Son olarak test verisiyle, hiçbir parametre değiştirmeden, bu K için başarı ölçülür ve tarafsız (önyargısız) olarak raporlanır. Örneğin "K = 5 için doğruluk (accuracy) 0,92".

Verinin eğitim, doğrulama ve test için nasıl ayrılacağı, birkaç hafta sonra model seçimi (model selection) üzerine yapılacak ayrı bir derste anlatılacak. Şimdilik bilinmesi gereken, farklı K değerleri için sistemin başarısını ölçüp en uygun K'yi buna göre belirlemenin en doğru çözüm olduğudur.

Ders sonuna doğru bir öğrenci K'nin nasıl belirlendiğini tekrar sordu.

Soru: K'yi nasıl 5 belirlediğimizi tekrar söyleyebilir misiniz? **Cevap:** Doğrulama örneklerinin her birini K = 1 için sınıflandırıp başarıyı ölçüyorsunuz. Sonra K = 3, 5, 7 için aynısını yapıyorsunuz. Hangi K için başarı en yüksekse onu kullanıyorsunuz. Sonra test örnekleriyle o K için başarıyı hesaplayıp tarafsız bir şekilde raporluyorsunuz. Öğrenci "yani K'nin değerini aslında deneyerek, sonuca göre belirliyoruz" dedi; hoca bunu onayladı.

> **Hocanın vurgusu:** Model seçiminde hep böyle yapılır. Çünkü probleminize göre hangi parametrenin daha iyi olacağı değişir. Yapay sinir ağlarında katman sayısını belirlerken de, karar ağacını budarken de hep deneyerek yaparsınız: Farklı parametrelerle modelin başarısını ölçer, en başarılı sonucu veren parametrelerle modeli kullanırsınız. K'yi ezbere belirleyemezsiniz.

Hoca sebebini de açıkladı: Probleme göre örneklerin dağılımı ve karar sınırlarının dağılımı değişir. Birinde sınırlar doğrusal, birinde doğrusal değildir; doğrusal olmayanlarda dağılım Gauss olabilir ya da olmayabilir; birçok farklı senaryo vardır. Örneğin bir problemde elinizdeki örnekler için K = 3 en yüksek başarıyı verirken, kredi kartı sahtekârlığı tespiti gibi bambaşka bir problemde K = 5 en iyi sonucu verebilir.

14. Özelliklere Ağırlık Verme

k-NN'de yapılabilecek bir iş daha, özelliklere ağırlık eklemektir. Bazı özellikleriniz diğerlerinden daha değerliyse, o özelliğe ağırlık ekleyerek mesafedeki etkisini artırabilirsiniz. Hoca bunun iki yolunu anlattı:

1. **Ağırlığı deneysel olarak belirlemek.** Daha önemli olduğunu düşündüğünüz özelliğe farklı ağırlık değerleri verip deneyerek hangisinde daha başarılı olduğuna bakarsınız.
2. **Ağırlığı doğrudan bir bağıntıyla eklemek.** Özelliklerin farkının karesinin bir bölüsü ağırlık olarak eklenir:

$$w_k = \frac{1}{(x_k - x_{new,k})^2}$$

Bilinmeyen örnekteki k. özellik ile bildiğiniz örnekteki k. özellik birbirine ne kadar benziyorsa, yani fark ne kadar küçükse, "bir bölü" olduğu için ağırlık o kadar büyük olur ve mesafeyi o

kadar çok etkiler. Bu yolda ağırlıkla deneme yaparak uğraşmanız gerekmez.

15. Karmaşıklık ve Tembel Öğrenme

Hoca öğrencilere k-NN'in karmaşıklığını sordu ve cevabı onlarla birlikte kurdu.

Soru (hocadan): Bilinmeyen örnek ile tek bir eğitim örneği arasındaki mesafe hesabının karmaşıklığı nedir? Öğrencilerden biri $O(n^2)$ dedi, biri $O(1)$ dedi. **Cevap:** Aslında $O(D)$'dir; D özellik sayısıdır. Öğrencinin dediği gibi, D çok büyük bir sayı değilse $O(1)$ de denebilir.

Hoca burada bir uyarı yaptı. $O(1)$ diyebilmek için D'nin oldukça küçük olması gerekir. Özellik sayısı 1000 bile olsa aslında küçüktür; ama on binler, yüz binler gibi büyüyebilen bir sayıysa D olarak bırakılmalıdır. Normalde örneklerde beklenen, özellik sayısının az olmasıdır.

Soru (hocadan): N tane eğitim örneği varsa bir en yakın komşuluğun (1-NN) karmaşıklığı nedir? Bir öğrenci "N çarpı D" dedi. **Cevap:** Evet, $O(N \cdot D)$. D küçük bir sabitse (10, 100 gibi) bu $O(N)$'dir. D büyükse N^2'ye kadar gidebilir; bu yüzden anlaşılırlık için $O(N \cdot D)$ yazmak daha doğrudur.

Hoca K en yakın komşu için karmaşıklığı $O(K \cdot N \cdot D)$ olarak verdi (bkz. Ders Sonrası Notlar).

15.1 Tembel öğrenme (lazy learning)

k-NN'e **tembel öğrenme** (lazy learning) denir. Çünkü bir şey öğrenmez. Eğitim örnekleri gelir ve orada durur. Yalnızca test aşamasında bilinmeyen örnek tek tek bütün eğitim örnekleriyle karşılaştırılır. Bu yüzden eğitim örneği sayısı az ise hızlıdır, fazlaysa yavaşlar: N ne kadar büyürse o kadar çok karşılaştırma yapılır.

Hoca, k-NN'in ilk kullanılmaya başlandığı eski dönemlerde, eğitim örneği sayısı 20-30-40 gibi küçük sayılarsa kullanılmasının uygun olduğunun söylendiğini anlattı. Eski kaynaklarda bunu görebilirsiniz. Ama bugün bu kadar az örnekle genellemeye gitmek zor olduğu için k-NN'i daha çok örnekle kullanmak istiyoruz. O zaman süreden kazanmak için ne yapabiliriz?

Soru (öğrenciden): Eğitim kümesi için hesaplamayı bir kez yapıp hafızada tutsak, her seferinde hesaplama yapmasak? **Cevap:** Bunu yapamazsınız, çünkü bilinmeyen örnek her seferinde yeni bir örnektir. Eğitim örnekleriyle önceden hesaplanacak bir şey yoktur. Bilinmeyen örnek geldiğinde onun bütün örneklere mesafesine bakılır. Sonra mesafesi en küçük K örnek seçilir; bunun için de K'ıncı en küçüğü bulma ya da ilk K elemanı sıralama gibi bir iş yapılır. Yani yüklü bir iş vardır.

Soru (hocadan): Burada işi zorlaştıran nedir? Bir öğrenci "N'in karmaşıklığı mı?" dedi. **Cevap:** N'in büyüklüğü, yani örnek sayısı. O zaman örnek sayısı azaltılmaya çalışılabilir. Soru, örnek sayısının nasıl azaltılacağıdır.

> **Hocanın vurgusu:** "500 örneğim var, bunların 100 tanesini rastgele seçeyim" demek iyi bir şey değildir. Rastgele seçimle çok işe yarayan örnekler de gidebilir. Mantıklı bir eleme yöntemi gerekir.

Bu noktada derse kısa (yaklaşık 5 dakikalık) bir ara verildi.

16. Voronoi Diyagramı ve Gereksiz Örnekler

Aradan sonra hoca, hangi örneklerin gereksiz olduğunu anlamak için iki özellikli bir çizim yaptı.

Eğitim setindeki her örnek için, komşu örneklerle arasında, aralarındaki uzaklığı eşit bölen bir sınır çizildiğini düşünelim. Her örnek kendi bölgesine sahip olur. Bu yapıya **Voronoi diyagramı** denir. Hoca gündelik bir örnek verdi: Baz istasyonlarının kapsama alanları Voronoi diyagramına göre belirlenir. İki baz istasyonunun tam ortasından geçen bir sınır olduğu varsayılır; sınırın bir tarafı bir istasyonun, diğer tarafı öbür istasyonun kapsama alanıdır. Bilgisayar ağları derslerinde bunu görmüş olabilirsiniz. Burada da her örneğin, diğer örneklere göre karar sınırı bu şekilde belirlenir.

Hoca sonra soldaki bölgede A sınıfından, sağdaki bölgede B sınıfından örneklerin bulunduğu bir çizim yaptı ve bilinmeyen bir örneği A tarafına koydu.

Soru (hocadan): Bilinmeyen örnek buradayken, uzaktaki (sağdaki) B örneklerinin sınıflandırmaya bir katkısı var mı? Bir öğrenci önce ortalamaya etkisi açısından yorum yaptı; hoca ortalamayla ilgilenmediğimizi söyleyip soruyu daha fazla örnekle yeniden çizdi. Öğrenciler "yok" dedi. **Cevap:** Yok. Çünkü o tarafta hiç A sınıfından örnek yoktur. O bölgedeki B örneklerinin sınıflandırmaya katkısı yoktur; bunları iptal edebilirsiniz.

Bu örneklerin katkısı ancak çevrelerinde başka sınıftan örnekler olsaydı olurdu. Örneğin o bölgede A sınıfından örnekler de devam etseydi, B örnekleri iki sınıfın karışmasını önlemek için gerekli olurdu. Aynı mantıkla, A bölgesinin içinde, çevresinde hiç B örneği olmayan A örnekleri de gereksizdir.

Gözle bakınca "şu ve şu örnekler gereksiz" demek kolaydır. Ama hangilerinin eleneceğine sayısal olarak karar vermek gerekir. Voronoi diyagramı burada yol gösterir: Bir örneğin Voronoi komşularının hepsi kendisiyle aynı sınıftaysa, o örneği kaldırmanın bir sakıncası yoktur. Çünkü o civarda tehlike yaratacak başka sınıftan bir örnek yoktur. Voronoi komşusu, K = 1 için en yakınındaki komşular demektir. Sınıf sınırındaki örnekler ise korunmalıdır.

Hoca rahat anlaşılsın diye bunu hep iki sınıf üzerinden gösterdiğini, ama k-NN'de sınıf sayısının sınırlı olmadığını vurguladı: 3 sınıf da, 5 sınıf da olabilir; k-NN'in kaç sınıfla çalışacağı konusunda bir sınırı yoktur.

17. Düzenlenmiş Örnek Tabanlı Öğrenme: Artımlı Silme ve Artımlı Büyütme

Örnekleri elemek için kullanılan yöntemlerden biri **düzenlenmiş örnek tabanlı öğrenme** (edited instance-based learning) yaklaşımıdır. Amaç gereksiz örneklerden kurtulmaktır. Hoca bunun iki yönünü anlattı.

17.1 Artımlı silme (incremental deletion)

Bu yöntem yukarıdan aşağıya (top-down) çalışır:

1. Elinizdeki bütün eğitim örnekleriyle başlarsınız.
2. Her seferinde bir örneği, sanki bir test örneğiymiş gibi ele alır, kalan örneklerle k-NN ile sınıflandırırsınız.
3. Örnek doğru sınıflandırılıyorsa, çevresinde hep kendisiyle aynı sınıftan örnekler vardır. Bu örneği silersiniz.
4. Yanlış sınıflandırılıyorsa, örnek sınırdadır ve korunması gerekir.

Hoca bunu üç sınıflı (A, B, C) bir çizimle gösterdi. Çevresi tamamen B sınıfı örnekleriyle çevrili bir B örneğini silmenin kimseye zararı yoktur. Ama hem A hem B sınıfından komşuları olan sınırdaki bir örnek silinirse yanlış olur. Gerçekten de K = 5 ile sınıflandırıldığında bu örnek yanlış sınıflanır (A'ya daha çok benzediği görülür); demek ki silinmemesi gereken bir örnektir.

Bir öğrenci K = 5 için komşulardan bazılarının farklı sınıftan olduğu durumu sordu.

Soru: K'yi 5 seçtik. Komşulardan biri maviden (örneğin kendi sınıfından), ikisi başka sınıftan çıktı, ama oylama yine doğru sınıfı veriyor. Bunu silecek miyiz? **Cevap:** Siliyorsunuz. Burada önemli olan K'yi kaç seçtiğinizdir. Silme işlemi yapacaksanız K çok küçük seçilmemeli, 5 gibi bir değer olmalı ki silindiğinde bir hataya sebep olmasın.

Soru: Silmesek daha mantıklı değil mi? Beş komşunun hepsi aynıysa silelim, biri bile farklıysa sınırda diyelim ve silmeyelim? **Cevap:** Önce modelinizi oluşturdunuz: K'nin kaç olacağına karar verdiniz, diyelim K = 5 en iyi sonucu verdi. Model hazır. Şimdi eğitim örneği olmasına gerek olmayan örneklere bakıyorsunuz ve K en yakın komşusuyla aynı sınıflanmayan noktaları ayırıyorsunuz. Yöntemin temel prensibi doğru sınıflandırılmadır; doğru sınıflanıyorsa silmek yeterlidir. Ama çok hassas davranmak istiyorsanız "beş komşusu birden aynı değilse koruyacağım" da diyebilirsiniz. İkisini de deneyip başarının nasıl değiştiğine bakın. Modeli bir kez kuracaksınız; deneyerek yapın. Amaç gereksiz örnekleri atıp N'i küçültmektir. Rastgele seçseydiniz belki çok işe yarar bir sınır örneğini harcayacaktınız.

Hocanın vurgusu: k-NN, örnek sayısının çok olduğu durumlar için tercih edilen bir yöntem değildir. Daha çok eğitim örneklerinin az olduğu ve çoğaltılamadığı durumlarda uygundur; örneğin hastalık teşhisi için görüntülerle çalışırken bulabildiğiniz görüntü sayısı yüzü geçmiyordur. Binlerce, on binlerce örnek varsa iki açıdan iyi değildir: Eğiterek bir model oluşturmazsınız ve test aşamasında işlem yükünüz çok fazladır. Özellik sayısı da fazlaysa, ortada model olmadığı için her örnekle tek tek karşılaştırma yapmanız gerekir.

17.2 Artımlı büyütme (incremental growth)

Bu yöntem aşağıdan yukarıya (bottom-up) çalışır ve öncekinin tersidir:

1. Boş bir kümeyle (bellekle) başlarsınız.
2. Eğitim örneklerini tek tek alır, bellekteki örneklerle k-NN ile sınıflandırırsınız.
3. Doğru sınıflandırılıyorsa eklemezsiniz (atarsınız).
4. Yanlış sınıflandırılıyorsa, korunması gereken, sınır oluşturabilecek bir örnek olduğu için belleğe eklersiniz.

Hoca bunu iki sınıflı bir çizimle, bellekteki örnekleri siyah renkle göstererek anlattı. İlk seçilen örnek belleğe girer. İkinci örnek büyük ihtimalle yanlış sınıflanır ve o da korunur. Sonraki örneklerden doğru sınıflananlar atılır, yanlış sınıflananlar eklenir.

Burada bir başlangıç problemi vardır. K = 3 ise, boş bellekle başlayınca ilk örnekler için üç komşu bile yoktur. Bu yüzden başlangıçta her sınıftan K kadar (örneğin K = 3 için 3'er) örnek seçilerek başlanmalıdır. Hoca çizimde K = 3 için üç örnekle başlayıp üzerine ekleye ekleye gitti: Doğru sınıflanan bir örnek atıldı; en yakın iki komşusu diğer sınıftan olduğu için yanlış sınıflanan bir örnek korundu; çevresi tamamen kendi sınıfıyla dolu olan bir örnek doğru sınıflandığı için atıldı.

Bu yöntemin bir sıkıntısı vardır: Başlangıç örnekleri sınırdan uzak bölgelerden seçilirse, yöntem o bölgelerdeki örnekleri mecburen alır. Yani gereksiz örnekleri belleğe alma ihtimali vardır. Artımlı silmede böyle bir sorun yoktur, çünkü bütün kümeyle başlanır ve bir örneğin nerede olduğu baştan bellidir.

Hoca iki yöntemin de kullanılabilir olduğunu söyledi.

18. K Boyutlu Ağaç (k-d Tree)

Örnek sayısının yükünü azaltmanın üçüncü yolu **K boyutlu ağaçtır** (k-dimensional tree, k-d tree). Hoca bunun biraz daha zor bir yöntem olduğunu, belki veri yapıları derslerinde görmüş olabileceğimizi söyledi.

Normalde k-NN'de bilinmeyen örneğin bütün eğitim örnekleriyle mesafesini tek tek ölçeriz. k-d tree'de ise önce eğitim örnekleriyle bir ağaç kurulur. Bilinmeyen örnek bu ağaç üzerinde ilerleyerek hangi bölgede (region) olduğunu bulur. Örneğin iki özellik varken ağaçta "$x_1 > 6$ mı?" diye sorulur. Cevaba göre uzayın bir yarısındaki örneklerle hiç karşılaştırma yapılmaz.

18.1 Ağaç nasıl ilerler?

Hoca önce kendisi bir çizim yaptı (eksenlerde 0, 2, 4, 6, 8, 10 değerleri), sonra paylaşımını değiştirip hazır bir örnek gösterdi. Örnekte A, B, C, D, E, F, G, H, I, J gibi harflerle gösterilen noktalar eğitim örnekleriydi ve sağ tarafta bu örneklerden kurulmuş ağaç vardı.

Örnek 8: Gösterilen k-d tree üzerinde arama

- Kök düğüm F: $x_1 > 6$ mı? F noktası, $x_1 = 6$ boyunca bir sınır çizerek uzayı x_1'in 6'dan büyük ve küçük olduğu iki tarafa böler.
- $x_1 \leq 6$ tarafındaki ilk düğüm C: $x_2 > 10$ mu? C uzayı bu kez yatay olarak böler. $x_2 > 10$ tarafında A ve B, $x_2 \leq 10$ tarafında D ve E vardır.
- Alt düğümler de benzer şekilde devam eder: "$x_2 > 4$ mü?", "$x_1 > 3$ mü?" gibi.
- Sağ tarafta da benzer düğümler vardır; hoca çizimde $x_2 > 5$ için H, $x_1 > 9$ için G noktalarını örnek verdi.
- Bilinmeyen örnek $(2, 3)$ gelsin:

  - $x_1 > 6$ mı? Hayır.
  - $x_2 > 10$ mu? Hayır.
  - $x_2 > 4$ mü? Hayır.
  - Yaprağa ulaşıldı. Sınıflandırma burada sona erer.
- Hoca bu yapraktaki örneklerin (anlatımında "E ve F") hangi sınıfa ait olduğuna göre bilinmeyen örneğin sınıfının belirleneceğini söyledi.
- Bu örnek için I, J, H, G gibi diğer örneklerle tek tek mesafe hesaplanmasına gerek yoktur. Bölgesi bellidir.

Bu yapı bir karar ağacına benzer: Yaprağa geldiğinizde bir sınıf elde edersiniz ve sınıflandırmayı yalnızca ağaç üzerinde ilerleyerek yaparsınız.

18.2 Karar ağacından farkı

Soru: Karar ağaçlarından belirgin bir farkı var mı? **Cevap:** Karar ağaçlarında ağaç özelliklerin durumuna, olasılıklarına (bilgi kazancına) göre kuruluyordu. Düğümler özellikleri içeriyordu: sıcaklık, rüzgâr gibi. Burada ise özellik seçimi yoktur. Ağaç yalnızca mesafeye göre kurulur ve her düğüm bir örneği içerir. İlerleme örneklerin konumuna göre yapılır, çünkü buradaki derdimiz mesafedir. Hoca gösterilen örneğin Manhattan mesafesiyle yapıldığını da belirtti.

Soru: Burada K'nin bir önemi yok değil mi? 3 ya da 5 seçmemiz bir şey ifade etmeyecek mi? **Cevap:** Burada da önce K'yi seçersiniz ve modele karar verirsiniz. Örneğin K = 3 için eğitim örneklerini ağaca aktarırsınız.

18.3 Karmaşıklık ve sınırlar

k-d tree bir ikili ağaçtır (binary tree). Her adımda bir grup örnek elendiği için, eğitim kümesinde N örnek varsa, ortalama durumda (average case) arama $O(\log N)$ karmaşıklıkla yapılır. Ağaç iyi kurulmazsa en kötü durumda yine $O(N)$ karmaşıklığa düşülür.

Bu yöntemin önemli bir sınırı vardır: Özellik sayısı arttığında böyle bir ağacı kurmak çok zor ve karmaşık bir işlemdir. İki özellikle gözle bakınca iş kolaydır, ama özellik sayısı arttıkça ağaç kurmak zorlaşır. Bu yüzden özellik sayısının fazla olduğu durumlarda tercih edilen bir yöntem değildir.

Soru: Kökte F örneğinden ($x_1 = 6$) başlamamızın sebebi, en ortadaki örnek olması mı? **Cevap:** Hoca bu ayrıntıya girmek istemediğini söyledi. Hangi düğümün kökte, hangisinin sonra geleceğine branch and bound yöntemiyle karar verildiğini, burada karar ağacındaki gibi bilgi kazancı (information gain) kullanılmadığını belirtti. Ağacın öncelik kuyrukları (priority queue) ile kurulduğunu, bunun tamamen algoritmik bir konu olduğunu ve makine öğrenmesinden biraz farklı olduğunu söyledi. İlgilenen olursa ağacın nasıl kurulduğunu ayrıca anlatabileceğini ekledi (bkz. Ders Sonrası Notlar).

> **Hocanın vurgusu:** k-d tree'nin mantığını kavram olarak anlamanız yeterlidir. Eğitim örnekleri mesafe bilgisine göre bir ağaca yerleştirilir. Örneğin "$x_1 > 6$" dendiği anda bir grup örnek elenmiş olur ve onlarla hiç işlem yapılmaz. Bu da ortalama durumda karmaşıklığın $O(\log N)$'e inmesini sağlar.

Dersin Toparlaması

Bu derste parametrik olmayan, tembel bir öğrenme yöntemi olan K en yakın komşu işlendi. Eğitimde yalnızca etiketli örnekler saklanır; bilinmeyen örnek geldiğinde bütün eğitim örneklerine mesafesi hesaplanır, en yakın K örnek seçilir ve çoğunluk oylamasıyla sınıf belirlenir. Yöntemin iki temel kararı K'nin değeri ve mesafe ölçüsüdür. Mesafe için genelde Öklid, hız gerekiyorsa Manhattan, kategorik özelliklerde Hamming, dokümanlarda kosinüs benzerliği kullanılır. Mesafe tabanlı her yöntemde özellikler önce min-max normalizasyonu ya da standartlaştırma ile aynı ölçeğe getirilmeli, test örneği de aynı dönüşümden geçirilmelidir.

K tek sayı ve genelde 1 ile 7 arasında seçilir; en iyi değer doğrulama kümesinde farklı K'ler denenerek bulunur ve başarı test kümesiyle tarafsız olarak raporlanır. Yöntemin zayıf yanı test anındaki işlem yüküdür ($O(N \cdot D)$). Bu yükü azaltmak için Voronoi mantığıyla gereksiz örnekleri eleyen artımlı silme ve artımlı büyütme yöntemleri ile arama alanını daraltan k-d tree

anlatıldı. Hoca, verinin eğitim, doğrulama ve test için nasıl bölüneceğini ilerideki model seçimi dersinde, kategorik özelliklerin sayıya çevrilmesini (one-hot encoding) ise ileriki haftalarda anlatacağını söyledi.

Ders Sonrası Notlar

Ders sonrası not: Hoca Hamming mesafesinde eşit özelliğe 1, farklı özelliğe 0 verdi ve bunları topladı. Standart tanım bunun tersidir: Farklı olan özellik 1, eşit olan 0 sayılır ve mesafe, farklı olan özelliklerin sayısıdır. Derste hesaplanan değer aslında eşleşen özellik sayısı, yani bir benzerliktir. Standart tanımla (Ankara, İzmir, İstanbul) ile (Ankara, Diyarbakır, Muğla) arasındaki Hamming mesafesi 2, (Ankara, İzmir, Muğla) ile arasındaki ise 1'dir.

Ders sonrası not: Öklid örneğinde hoca $\sqrt{42}$ için önce "6,6", sonra "6,4 falan" dedi. Doğrusu $\sqrt{42} \approx 6{,}48$'dir.

Ders sonrası not: Hoca k-NN'in karmaşıklığını $O(K \cdot N \cdot D)$ olarak verdi. Daha yaygın ifade şudur: Bütün eğitim örneklerine mesafe hesabı $O(N \cdot D)$, en yakın K örneği seçmek yığın (heap) ile $O(N \log K)$ ya da seçme algoritmasıyla $O(N)$'dir. Toplam genelde $O(N \cdot D)$ olarak verilir; K, mesafe hesabıyla çarpılmaz. Hocanın "D büyükse N^2'ye kadar gider" ifadesi de ancak D'nin N mertebesinde olduğu durum için geçerlidir.

Ders sonrası not: Artımlı silmeyi anlatırken hoca bir yerde "doğru sınıflandırmıyorsa silebilirsiniz" dedi. Kuralın doğrusu, aynı anlatımın devamında da söylendiği gibi: Doğru sınıflanıyorsa silinir, yanlış sınıflanıyorsa korunur.

Ders sonrası not: Artımlı büyütmeyi çizerken hoca bir yerde "doğru sınıflandırılıyorsa koruyorsunuz" dedi. Kuralın doğrusu: Doğru sınıflanan örnek eklenmez, yanlış sınıflanan örnek eklenir.

Ders sonrası not: Hoca k-d tree'de kök düğümün branch and bound ile seçildiğini söyledi. Standart k-d tree kurulumunda bölme ekseni her seviyede sırayla değiştirilir (x_1, x_2, x_1, ...) ve genelde o eksendeki medyan nokta düğüm yapılır. Branch and bound ise ağaç kurulduktan sonra, en yakın komşuyu ararken hangi dalların budanabileceğine karar vermek için kullanılır.

Ders sonrası not: Derste "cosine distance" ve "cosine similarity" ifadeleri birbirinin yerine kullanıldı. Kosinüs benzerliği $\cos\theta$'dır ve büyük değer daha benzer demektir. Kosinüs mesafesi genelde $1 - \cos\theta$ olarak tanımlanır; k-NN'de mesafe olarak kullanılacaksa sıralama bu değere göre küçükten büyüğe yapılır.

Ders sonrası not: Hocanın andığı Mahalanobis mesafesi, kovaryans matrisi Σ ile $d(x, y) = \sqrt{(x-y)^T \Sigma^{-1} (x-y)}$ olarak tanımlanır.

Ders sonrası not: Özellik ağırlığı için verilen $w_k = 1/(x_k - x_{new,k})^2$ bağıntısı, fark sıfır olduğunda tanımsızdır. Yaygın kullanılan ağırlıklı k-NN'de ağırlık özelliklere değil komşulara verilir: Her komşunun oyu, bilinmeyen örneğe olan mesafesinin karesinin tersiyle ($1/d^2$) ağırlıklandırılır.

Ders sonrası not: Altyazıda "öplik distance", "Heming", "kosan similarity", "Varona'yı diyagramı", "Mahalonobis", "incremental glation" gibi geçen terimlerin doğru yazımı sırasıyla Euclidean (Öklid) distance, Hamming, cosine similarity, Voronoi diyagramı, Mahalanobis ve incremental deletion'dır.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.